CS612 Lecture 01 — In-Class Exercises

Advanced Algorithm Analysis, Correctness & Lower Bounds · Sat 26 Sep 2026

These exercises are ungraded. They are worked in class; unfinished parts are good preparation for PS1.

Opening — Believe the vendor?

A vendor makes three claims about a sorting library for records with 64-bit keys. For each, decide whether it is **impossible**, **possible under assumptions** (state them), or **not a meaningful claim**, and name the single assumption that decides it.

1. "Our comparison sort needs only $3n$ comparisons on any input."
2. "We sort 10^8 records in $O(n)$ time."
3. "Our sort is $O(n \log n)$, so it is always faster than your $O(n^2)$ insertion sort."

A — True, false, or needs qualification?

For each statement write T, F, or the qualification that makes it precise.

1. Merge sort is $O(n^2)$.
2. Quicksort is $O(n \log n)$.
3. Insertion sort is $\Omega(n^2)$.
4. A $\Theta(n \log n)$ worst-case algorithm is always faster than a $\Theta(n^2)$ worst-case one.
5. Counting sort breaks the $\Omega(n \log n)$ sorting lower bound.
6. Hash table lookup is $O(1)$.
7. Dynamic-array append is $O(1)$ on average.

B1 — Find the flaw

Specification: $A[0..n-1]$ is sorted non-decreasingly and $A[0] \le x$. Return the largest index i with $A[i] \le x$.

```
LAST-LE(A, n, x)
  lo = 0 ; hi = n − 1
  while lo < hi
      mid = ⌊(lo + hi) / 2⌋
      if A[mid] ≤ x
          lo = mid
      else
          hi = mid − 1
  return lo
```

A student's proof: "Invariant: $A[lo] \le x$, and every $j > hi$ has $A[j] > x$. Both branches preserve it. The interval $[lo, hi]$ shrinks every iteration, so the loop terminates, and at exit $lo = hi$ is the answer."

1. Which sentence of the proof is false?
2. Give a concrete input on which the algorithm fails.

3. Fix the algorithm with a minimal change and prove termination with an explicit variant.

B2 — Invariant practice (if time permits)

Write a loop invariant and variant for the following algorithm, then prove it returns $\max(A[0..n-1])$ for $n \geq 1$.

```
MAX(A, n)
  m = A[0] ; i = 1
  while i < n
     if A[i] > m
        m = A[i]
     i = i + 1
  return m
```

B3 — Counterexamples

Give the smallest counterexample you can for each claim.

1. "For coins $\{1, 3, 4\}$, repeatedly taking the largest coin that fits minimises the number of coins."
2. "Choosing the interval that *starts* earliest maximises the number of pairwise non-overlapping intervals."
3. "Choosing the interval with the *fewest overlaps* with other intervals maximises the number of non-overlapping intervals." (Harder. *Hint:* an optimal solution of four intervals in a row, one "cheap" interval straddling the middle two, and stacks of duplicates to make the optimal intervals look expensive.)

C — Lower bounds

1. **Searching.** Prove that any comparison-based algorithm that searches a sorted array of n distinct keys (returning the index of x, or the position where x would be inserted) needs at least $\lceil \log_2(n+1) \rceil$ comparisons in the worst case.
2. **Merging.** Two sorted lists $a_1 < \cdots < a_n$ and $b_1 < \cdots < b_n$ are to be merged.
   - (a) Give the decision-tree lower bound.
   - (b) Show that an adversary can force $2n - 1$ comparisons. *Hint:* consider the final order $a_1 < b_1 < a_2 < b_2 < \cdots < a_n < b_n$.
3. **Reduction.** Prove that any comparison-based algorithm that builds a binary search tree from n unsorted keys makes $\Omega(n \log n)$ comparisons in the worst case.
4. **Too good to be true.** Does a comparison-based priority queue exist with worst-case $O(1)$ *insert* and worst-case $O(1)$ *extract-min*? Justify.

Experiment (take-home, optional)

Compile and run `Student_Resources/comparison_counts.cpp` (`g++ -std=c++20 -O2`).

1. Compare merge sort's comparison count with $\lceil \log_2 n! \rceil$. Derive merge sort's worst-case count for $n = 2^k$ and explain the gap.
2. Explain why insertion sort's count is close to $n^2/4$ on random permutations.

3. Predict, from the formulas, the value of K at which counting sort stops beating `std::sort` for $n = 2 \times 10^6$. Compare with the measured table and give two reasons why the prediction and the measurement differ.